Mikhail Razakov

Static Analysis / Program Analysis Engineer — Roslyn · LLVM

Moscow / remote · razakov.mikhail@outlook.com · Telegram · GitHub

PROFILE

I develop static program analysis tools in C#/.NET and C++/LLVM. Since October 2026, I have worked on symbolic computation for SharpChecker at ISP RAS. My independent projects cover a Roslyn CFG-based null-state analyzer, interprocedural effect analysis for an LLVM transformation, and memory-dependence analysis. I use regression tests and reference implementations to check correctness.

TECHNICAL SKILLS

Languages and tooling: C#, .NET, Roslyn, C++23, LLVM 22 / LLVM IR, APInt, GNU C17, Python, Git, Linux

Analysis: CFG, worklist/fixed-point data flow, conditional-edge refinement, state joins, interprocedural effects, affine evolution and memory dependence

Correctness: Roslyn analyzer tests, LLVM Verifier, transformation-legality checks, regression and differential tests, independent reference oracles

EXPERIENCE

ISP RAS · SharpChecker · October 2026 — present

- Working on symbolic computation for C#/.NET static program analysis.

MCST · Compiler Engineering Internship · July — August 2026

- Worked on experimental LLVM LICM, checking loop structure, memory accesses and side effects before transformations; compared before/after behavior with Python.

ENGINEERING PROJECTS

DerefAfterNullAnalyzer

Built a Roslyn analyzer for potentially unsafe dereferences after null checks. Fixed-point CFG data flow refines states on conditional edges, merges at joins, and handles loops; it accounts for assignments and ref/out. Passed 18/18 tests on a pinned .NET 10 Git revision. Separate from SharpChecker; no alias/points-to or interprocedural analysis.

LLVM Interprocedural Global IV Optimization

An experimental LLVM pass promotes selected integer-global accesses within loops. Analysis of call effects and affine evolution precedes the separate PromotionLegality gate and IR rewriting. Passed 29/29 regression scenarios on LLVM 22.1.8: structural checks, LLVM Verifier, idempotence and selected execution comparisons; unsupported cases are conservatively rejected.

PS-form Memory Dependence Analyzer

Analyzes overlapping memory accesses across loop iterations using exact counting for supported affine forms and modular/GCD checks for selected disjointness cases. Returns conservative maybe when a proof is unavailable. Built for an MCST technical selection; not a comprehensive production analyzer.

EDUCATION

HSE University — Software Engineering

RECOGNITION

2026 · MCST LLVM/C++ selection: According to organizer correspondence: 5.0/5.0 and 104/104 official tests among 49 submitted solutions.

2025–2026 · MEPhI Junior: First-degree diplomas in the Information Technologies section in 2025 and 2026.

2026 · LangDev — UniversalToolchain: Delivered a talk on 9 October 2026; the organizer published the recording. — Talk recording